LINUX / SYSTEMS

# Performance Tuning Visual Guide

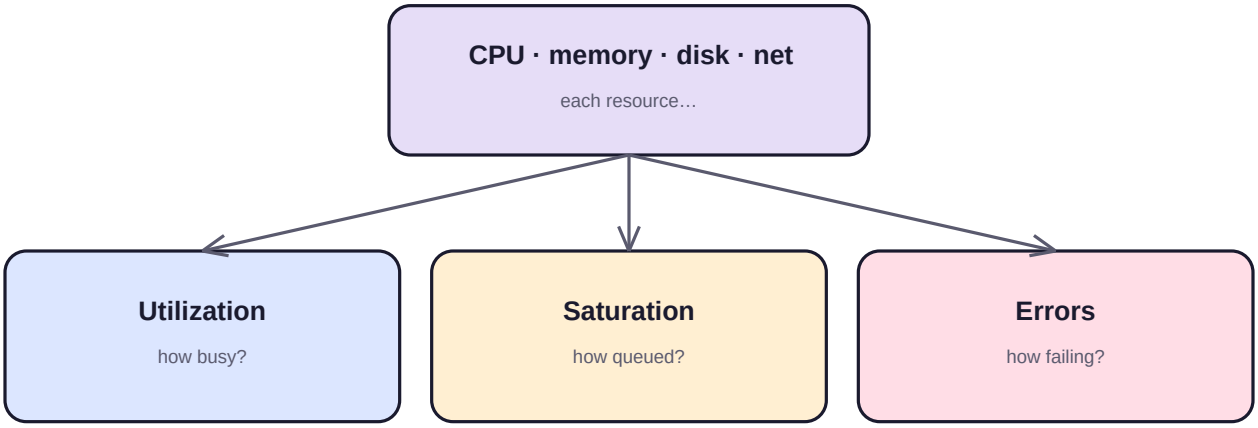
The USE method, CPU/memory/IO profiling, perf, and flame graphs.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/6

# The USE Method: Check Every Resource

For each resource ask: Utilization, Saturation, Errors.




| Resource | Utilization | Saturation | Errors |
|---|---|---|---|
| CPU | %user+sys | run queue | — |
| memory | used/total | swap/paging | OOM kills |
| disk | %util | await queue | io errors |
| network | throughput | drops | retransmits |

USE kills the #1 tuning mistake: optimizing the wrong resource. A saturated disk looks like a slow CPU until you check await.

STEP 2/6

# CPU: Where Do Cycles Go?

user vs system vs steal vs idle — then per-process.

```
$ mpstat -P ALL 1        # per-core: %usr %sys %iowait %steal %idle
$ pidstat -u 1           # per-process CPU
$ top → press 1           # per-core view
$ perf top               # live: which FUNCTIONS burn cycles
```

- High %iowait is NOT a CPU problem — the CPU is idle waiting on disk. Tune storage.
- %steal on a VM means the hypervisor took your cycles — noisy neighbor or overcommit.
- High %sys with lots of context switches → too many tiny syscalls or lock contention.

STEP 3/6

# Memory: Used Isn't Used

Linux eats free RAM for cache — that's a feature.

```
$ free -h
              total   used   free  shared  buff/cache available
Mem:           31Gi   4.2Gi   2.1Gi   1.0Gi       24Gi       25Gi
$ vmstat 1   # si/so = swap in/out — ANY sustained so = pain
$ dmesg | grep -i oom   # did the OOM killer visit?
```

- Watch 'available', not 'free' — buff/cache is instantly reclaimable.
- Swap OUT (so) under load = thrashing. Swap used at idle = harmless old pages.
- OOM killer picks by oom_score — tune with /proc/<pid>/oom_score_adj for sacrificial vs sacred.

STEP 4/6

# Disk & Network I/O

Latency first, throughput second.

```
$ iostat -x 1      # await (latency), %util, avgqu-sz per device
$ iotop -o         # which PROCESS is doing the I/O
$ ss -i            # per-socket retransmits, rtt
$ nstat | grep -i drop   # packet drops = saturation signal
```

- %util at 100% with low await = busy but fine. High await = the queue is the problem.
- iotop -o shows only processes actually doing I/O — skip the idle noise.
- Network retransmits + drops together = congestion; retransmits alone can mean a flaky peer.

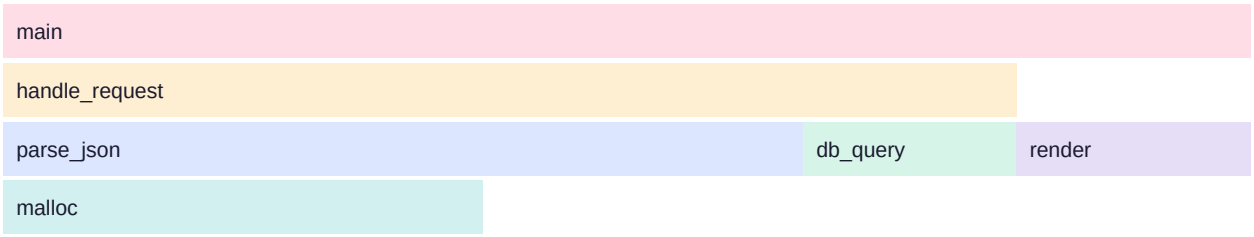
STEP 5/6

# perf & Flame Graphs

Sample the stack, draw the truth.

```
$ sudo perf record -F 99 -a -g -- sleep 30   # sample stacks, 30s
$ sudo perf report                          # who burned cycles
# flame graph (brendangregg/FlameGraph):
$ sudo perf script | stackcollapse-perf.pl | flamegraph.pl > out.svg
```

flame graph anatomy:



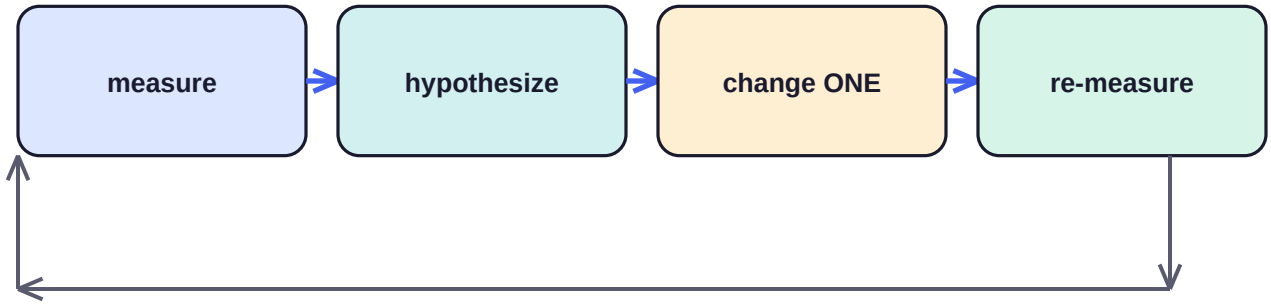

x-axis = time (width = cost)  y-axis = stack depth — the WIDEST plateau is your bottleneck

Read flame graphs by width, not height: the widest box at any level is where time goes. Ignore narrow towers — they're cheap.

STEP 6/6

# The Tuning Loop

Measure → hypothesize → change ONE thing → re-measure.




- Baseline FIRST: no 'before' numbers means no proof the change helped.
- One variable at a time — two changes and a win teaches you nothing reproducible.
- Tune the BOTTLENECK only (flame graph / USE). Optimizing idle resources is theater.
- Document the knob, the before/after, and the rollback — future-you is on call.

STEP ✓I✓

# Key Takeaways

## Performance Tuning

- USE method per resource: Utilization, Saturation, Errors — before touching any knob.
- CPU: %iowait is a disk problem; %steal is a hypervisor problem; %sys spikes mean syscall storms.
- Memory: watch 'available' not 'free'; sustained swap-out is the red alert.
- perf + flame graphs turn 'it's slow' into 'this function costs 40%'. Read by width.
- Tuning loop: baseline → one change → re-measure. No baseline, no proof.

**Keep exploring**

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev